An Efficient CNN Accelerator Using Inter-Frame Data Reuse of Videos on FPGAs

Shengzhao Li, *Graduate Student Member, IEEE*, Qin Wang, *Member, IEEE*, Jianfei Jiang, *Member, IEEE*, Weiguang Sheng, *Member, IEEE*, Naifeng Jing, *Senior Member, IEEE*, and Zhigang Mao, *Member, IEEE*

***Abstract*— Convolutional neural networks (CNNs) have had great success when applied to computer vision technology, and many application-specific integrated circuit (ASIC) and field-programmable gate array (FPGA) CNN accelerators have been proposed. These accelerators primarily focus on the acceleration of a single input, and they are not particularly optimized for video applications. In this article, we focus on the similarities between continuous inputs in video, and we propose a YOLOv3-tiny CNN FPGA accelerator using incremental operation. The accelerator can skip the convolution operation of similar data between continuous inputs. We also use the Winograd algorithm to optimize the conv3 × 3 operator in the YOLOv3-tiny network to further improve the accelerator's efficiency. Experimental results show that our accelerator achieved 74.2 frames/s on ImageNet ILSVRC2015. Compared to the original network without Winograd algorithm and incremental operation, our design provides a 4.10× speedup. When compared with other YOLO network FPGA accelerators applied to video applications, our design provided a 3.13×–18.34× normalized digital signal processor (DSP) efficiency and 1.10×–14.2× energy efficiency.**

***Index Terms*— Convolutional neural network (CNN), field-programmable gate array (FPGA) accelerator, incremental operation, input similarity, video applications, Winograd algorithm.**

I. Introduction

IN RECENT years, deep learning methods have played an important role in computer vision for areas, such as image recognition, target tracking, and classification. Many studies have proposed high-performance application-specific integrated circuit (ASIC) [1]–[4] and field-programmable gate array (FPGA) [5]–[9] neural network accelerators. Due to the rapid development of computer vision algorithms, the depth and scale of deep neural networks (DNNs) are also increasing. Meanwhile, there is an increasingly larger need for video processing versus traditional static image processing due to emerging applications such as autonomous driving and monitoring. Advancements in algorithms and changes in requirements have led to critical challenges to the data processing capabilities of current hardware. This article proposes a convolutional neural network (CNN) accelerator for video target recognition using an FPGA that achieves higher resource efficiency and higher speed than other FPGA CNN implementations when processing images of videos.

The difference between processing a single image and sequential frames of an input video is minimal. Therefore, the computation and memory accesses performed on sequential frames can be redundant. Traditional strategies perform a complete computation for each frame, which ignores the similarity in live vision. Our work attempts to produce a CNN that skips redundant operations between adjacent frames in video to achieve better throughput and resource efficiency on an FPGA.

Recently, some articles [10]–[13] have designed neural network accelerators based on data similarity between video frames. Previous work has primarily used the similarity of data between sequential frames in two ways.

1) With the exception of the first frame, the accelerator only processes the differences in sequential frames. The resulting convolution of the differential input is superimposed on the result from the previous frame operation [10]–[11].
2) By capturing visual motion in sequential frames, the accelerator can use the results obtained from key frames to predict the results of other frames, thereby avoiding redundant operations on nonkey frames [12], [13].

However, most of the works related to video CNN accelerators are about algorithms and modeling analysis. There are some designs devoted to high-performance and high-resource utilization CNN accelerators on FPGAs [14]–[17]. These works primarily use pruning and quantification methods to improve resource efficiency for static image tasks. There are no accelerator designs that take advantage of the data similarity between video frames on an FPGA.

Capturing visual motion in sequential frames for real-time video applications requires additional processing modules, which is not conducive to improving resource efficiency on FPGAs. The design in this article uses the YOLOv3-tiny network with 8-bit quantization, and it adopts the incremental operation of interframe differential input. As a result, our design achieves higher digital signal processor (DSP) efficiency and performance

Manuscript received 11 October 2021; revised 20 December 2021 and 27 January 2022; accepted 12 February 2022. Date of publication 26 September 2022; date of current version 24 October 2022. This work was supported by the National Key Research and Development Program of China under Grant 2018YFA0701500. *(Corresponding author: Qin Wang.)*

Shengzhao Li, Jianfei Jiang, Weiguang Sheng, Naifeng Jing, and Zhigang Mao are with the Department of Micro/Nano Electronics, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: lsz701014@sjtu.edu.cn).

Qin Wang is with the National Key Laboratory of Science and Technology on Micro/Nano Fabrication, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: qinqinwang@sjtu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TVLSI.2022.3151788.

Digital Object Identifier 10.1109/TVLSI.2022.3151788

1063-8210 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

compared to other high-performance [14]–[17] and YOLO accelerators [18]–[20] in video application scenarios while running at a reasonable image processing frame rate. To improve the DSP utilization rate under different CNN operators, our design uses the Winograd algorithm to increase both throughput while using the same DSP resources and the multiplexing efficiency of the processing element (PE) array.

In conclusion, we make the following contributions.

1) Our work uses the incremental operation of interframe differential input to take advantage of the interframe similarity of video input, and it improves processing speed and resource utilization. In addition, we use different frames that ignore thresholds in different layers to increase speed while keeping the network accuracy.
2) A corresponding dataflow is designed for the incremental operation method, which balances the sparse irregularity of the activation value for the incremental operation and improves the utilization rate of the DSP on the FPGA.
3) Using the Winograd algorithm improves throughput under the same DSP consumption and simultaneously increases the multiplexing of the PE array of the YOLOv3-tiny operators conv1 $\times$ 1 and conv3 $\times$ 3. This greatly improves DSP efficiency.

Experimental results show that our accelerator achieved 74.2 frames/s on ImageNet ILSVRC2015 [29]. Compared to the original network without Winograd algorithm and incremental operation, our design provides a 4.10$\times$ speedup. When compared with other YOLO network FPGA accelerators applied to video applications, our design provided a 3.13$\times$–18.34$\times$ normalized DSP efficiency and 1.10$\times$–14.2$\times$ energy efficiency.

II. Algorithm

This section introduces both the incremental operation and Winograd algorithm used in neural networks. Our work uses incremental operation strategies to reduce redundant operations in real-time visual processing, and it uses the Winograd algorithm to improve the utilization efficiency of DSP. We also implement a high-efficiency accelerator for real-time video processing on an FPGA.

The DNN in the universal neural network accelerator repeats a large number of calculations for each video frame, which wastes computing resources. Some articles [21], [22] proposed a new DNN calculation method to eliminate the wasteful calculations when processing video. The key idea of incremental operation is to find an algorithm that can update previously saved results. Incremental operation only needs to find the differences between frames and then correct the saved results throughout the differing parts. The corrected result can be approximated to the original calculation using the complete CNN. This type of algorithm is much cheaper than the complete convolution of each frame. It can reduce many of the redundant operations and memory accesses compared with the original network.

Incremental operation can be applied to any pretrained DNN, so most common networks can be optimized. Therefore, it is feasible to use incremental operation in hardware, which can reduce the computational cost of processing video.

In recent years, the Winograd algorithm has been widely used to accelerate DNNs [23]–[25]. It is an effective method that can significantly reduce the computational complexity of convolution calculations. Compared with original convolution calculations, F(2 $\times$ 2, 3 $\times$ 3) of the Winograd algorithm reduces the convolutional arithmetic complexity by 2.25 times. The YOLOv3-tiny network includes both conv3 $\times$ 3 and conv1 $\times$ 1 operators, with the majority being conv3 $\times$ 3, and the Winograd algorithm can reduce the arithmetic complexity required by the conv3 $\times$ 3 operators. When the conv3 $\times$ 3 operator is implemented using the Winograd algorithm, four 3 $\times$ 3 size convolution kernel operations are converted into 4 $\times$ 4 matrix elementwise products. The operation after the conversion is very similar to that of conv1 $\times$ 1 operators, which can reduce the complexity of the module and improve the utilization efficiency of the PE array. In our design, the Winograd algorithm allows us to further reduce the number of calculations required for convolution operations and to improve the efficiency of DSP on the FPGA.

A. CNN Incremental Operation

We define the input similarity as the ratio of the convolutional layer input of the current frame to the same part of the convolutional layer input of the saved frame, which is the frame before the current frame. In video applications, most of our inputs use 32-bit floating-point numbers with low input similarity. Although many parts of the current frame are like those of the previous frame, there are still small differences between them. Due to the high precision of 32-bit floating-point numbers, any small changes are reflected in the difference. However, small changes in the input have a minimal impact on the convolution results, and we can ignore these small changes in the incremental operation.

The common network quantization algorithm can significantly increase the similarity between inputs, which is conducive to our design for reducing redundant operations with minimal impact on recognition accuracy.

For pretrained CNNs, linear quantization is a popular quantization method [26]. We linearly quantize the activation and weights in each layer. After the quantization is completed, we evaluate the accuracy of the quantized network. The basic formula for linear quantization used for each layer is as follows:

$$\text{data}_{\text{quant}} = \text{round}\left(\frac{\text{data}_{\text{in}}}{\text{step}_l}\right) * \text{step}_l. \tag{1}$$

In our design, the steps used to quantify the weights and activation are different for different layers. The step is calculated according to the weights of each layer and the range of the activation value. The weight range is determined by analyzing the weights of the pretrained network, and the activation range is determined by analyzing the training set; $\text{data}_{\text{quant}}$ is the cluster centroid that is closest to the value of the input data. Our work uses 8-bit quantization, and the step calculation formula for each layer is given as follows:

$$\text{step}_l = 2^{\text{floor}(\max(|\text{data}_{\text{in}}|))-7}. \tag{2}$$

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

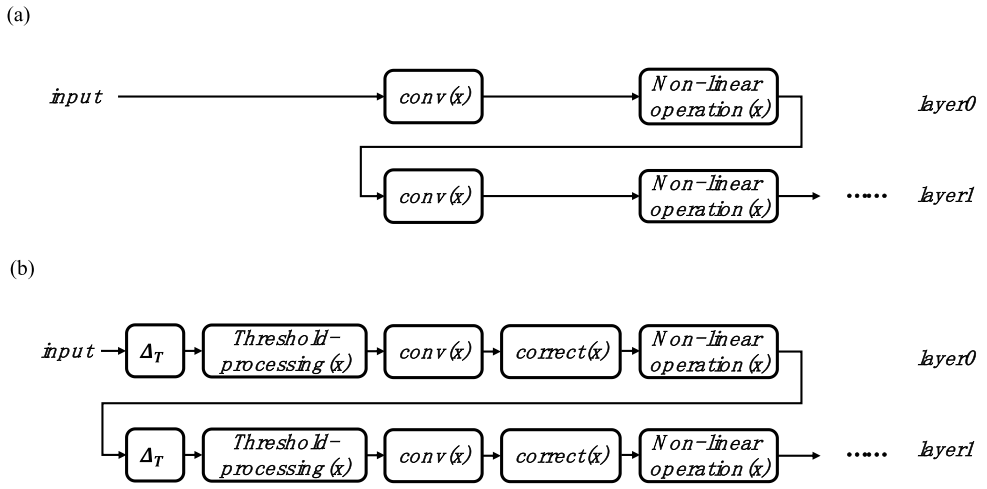


Fig. 1. (a) Original neural network calculation process. (b) Neural network calculation process with incremental operation.

TABLE I
ACCURACY OF THE NETWORK BEFORE AND AFTER QUANTIZATION

YOLOv3-tiny	Original	Quantized
Precision	32-bit floating	8-bit fixed
Task	Object detection	Object detection
mAP	0.742	0.731

Table I compares the accuracy of the quantized network on the ImageNet ILSVRC2015 dataset with the accuracy before quantization. We observe that the quantized network still has good accuracy.

The core idea of incremental operation using input similarity is that the convolution result of the previous frame will be saved and can be corrected to the result of the current frame. Considering that the convolution operation is a linear operation, the difference between the current frame input and the previous frame input is equal to the difference between the current frame convolution and the previous frame convolution. Therefore, we do not need to completely calculate the convolution of all frames. We only need to do the convolution operation on the difference between the input of the current frame and the saved frame and then correct the convolution result of the previous frame to obtain the convolution of the current frame. We can skip zeros and small values in the input difference because the input of the current frame is very similar to the input of the saved frame, and there are fewer values in their difference that influence the result. We define the sensitivity threshold as the value at which the input can be ignored. For the incremental operation, the calculation is performed using the following formula:

$$\text{Result}'_o = \text{Result}_o + \sum_{i=1}^{N} \left[\left(c'_i - c_i \right) * W_{io} \right]. \tag{3}$$

In a complete network, there are nonlinear operations, such as the ReLU and Maxpool operations, between each convolutional layer. Nonlinear operation cannot use incremental operations, so we need to perform ReLU and Maxpool after correcting the convolution results in each layer. Therefore, we need to insert the difference and result processing operations along with threshold processing in each layer. The network calculation process using incremental operation and the original neural network calculation process are shown in Fig. 1.

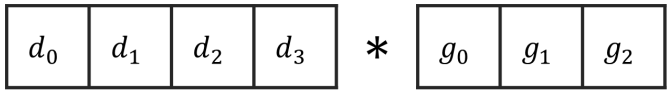


Fig. 2. One-dimensional Winograd algorithm.

B. Winograd Algorithm

Our design uses the F(2 × 2, 3 × 3) Winograd algorithm [23], which means that each Winograd processing unit will calculate 2 × 2 units of the 3 × 3 size convolution kernel. To illustrate how to perform the 2-D Winograd algorithm, we consider a 1-D F(2, 3) convolution operation. As shown in Fig. 2, the activation value is 1 × 4, and the convolution kernel is 1 × 3. The operation can be regarded as a 2 × 3 matrix multiplied by a 1 × 3 matrix, as shown in the following formula:

$$\begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} m_0 + m_1 + m_2 \\ m_1 - m_2 - m_3 \end{bmatrix} \tag{4}$$

where

$$\begin{aligned} m_0 &= (d_0 - d_2) g_0 \\ m_1 &= (d_1 + d_2) \frac{g_0 + g_1 + g_2}{2} \\ m_2 &= (d_2 - d_1) \frac{g_0 - g_1 + g_2}{2} \\ m_3 &= (d_1 - d_3) g_2. \end{aligned} \tag{5}$$

When the Winograd algorithm is applied to a 1-D convolution operation, it can be written in the following

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

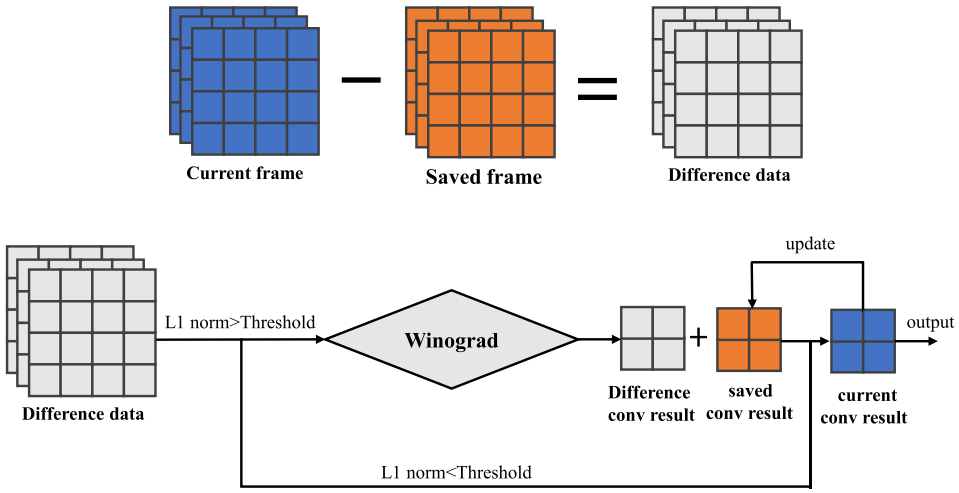


Fig. 3. Incremental operation process flow with Winograd algorithm.

matrix operation form:

$$Y = A^T\left[\left(GgG^T\right) \cdot \left(B^T \mathrm{dB}\right)\right]A. \tag{6}$$

For F(2 × 2, 3 × 3) used in this article, the matrix in formula (6) is given as follows:

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}$$
$$G = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}$$
$$A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}. \tag{7}$$

In the 2-D Winograd algorithm, g in F(2 × 2, 3 × 3) is a 3 × 3 filter, and d is a 4 × 4 image tile. In the original convolution operation, a 4 × 4 image tile will require a total of 2 × 2 × 3 × 3 = 36 multiplication operations. Through the Winograd algorithm, we can reduce the number of multiplications to 4 × 4 = 16 for a 4 × 4 image tile, thus reducing the arithmetic complexity by 36/16 = 2.25 times. The cost of the algorithm is that the convolution operation will have longer latency and more additional operations.

C. Incremental Operation With Winograd Algorithm

The incremental operation in Section II-A is optimized for the original CNN. For the CNN optimized with the Winograd algorithm, the incremental operation can still be used since the matrix operation is a linear operation. Due to the Winograd algorithm being based on a matrix operation of 4 × 4 image tiles, the difference and threshold processing in the incremental operation flow needs to be changed, as shown in Fig. 3.

The smallest unit of operation is a 4 × 4 image tile, and the smallest unit of difference in the conv3 × 3 layer is also a 4 × 4 image tile. In our work, the $L1$ norm of the difference between the image blocks of the previous and the next frame

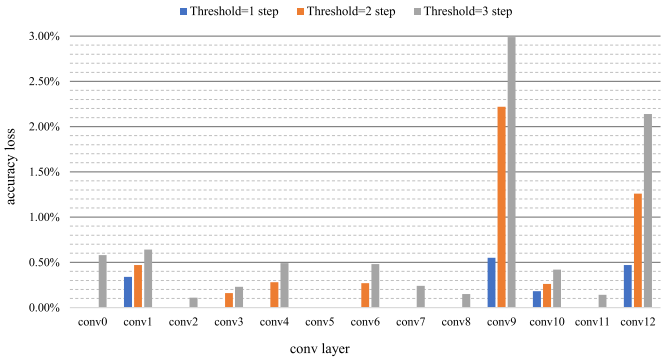


Fig. 4. Accuracy loss of conv layers with different thresholds.

is used to evaluate the similarity between the image blocks of the previous and next frames. The calculation process is given as follows:

$$\Delta_T = \sum_{i=n}^{n+3}\sum_{j=m}^{m+3}\left|d'_{ij} - d_{ij}\right|. \tag{8}$$

Here, d'_{ij} represents the activation in row i and column j of the stored frame and d_{ij} represents the activation in row i and column j of the current frame.

After the difference process is performed, threshold processing will determine the similarity between the modules. When the $L1$ norm of the difference is less than a certain threshold, the current frame image block is considered to be the same as the previous frame image block, and the Winograd operation can be skipped. When a higher $L1$ norm threshold is selected, our CNN will experience a drop in accuracy. Simultaneously, fewer image blocks need to be calculated. In order to make use of the similarity between consecutive inputs without accuracy loss, we use different thresholds in different layers. The accuracy loss of each layer with different thresholds is shown in Fig. 4.

Fig. 4 shows the accuracy loss of each layer when the threshold size is 1 step, 2 step, and 3 step, and the step is the quantization step. When the accuracy loss is less than 0.05%,

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THRESHOLD SETTING OF EACH LAYER, WHERE THE VALUE IN BRACKETS REPRESENTS A VALUE OF A SINGLE QUANTIZATION STEP

Conv layer	Threshold	Conv layer	Threshold
conv0	2 step(2^{-7})	conv7	2 step(2^{-5})
conv1	0 step(2^{-2})	conv8	2 step(2^{-5})
conv2	2 step(2^{-3})	conv9	0 step(2^{-3})
conv3	1 step(2^{-4})	conv10	0 step(2^{-2})
conv4	1 step(2^{-5})	conv11	2 step(2^{-5})
conv5	3 step(2^{-5})	conv12	0 step(2^{-3})
conv6	1 step(2^{-5})	/	/

we believe that this kind of threshold will not cause accuracy loss, which is not shown in Fig. 4. Also, there is no accuracy loss for 3 step of conv5 layer, and when the threshold of conv5 layer is 4 step, the accuracy loss is about 0.3%, so the threshold setting of conv5 layer is 3 step.

According to the accuracy loss of each layer, we set the threshold of each layer, which will not cause accuracy loss. The threshold setting is shown in Table II. The value in the brackets of Table II is the actual value of the quantization step. The accuracy of the network under the above threshold setting is 0.731, which is the same as the accuracy of the quantized YOLOv3-tiny without incremental calculation.

III. DATAFLOW

There are efficient dataflow designs for traditional CNN accelerators [16], [27], but they are only efficient in dense networks. These dataflows cannot take advantage of the video application characteristics, and they are not suitable for use with the Winograd algorithm. For our work, there are two important issues. First, the dataflow should be able to use the similarity between input frames to get better performance, i.e., the dataflow can skip the part of the input that does not need to be calculated after the difference. Note that dataflows with this characteristic are similar dataflows for sparse CNNs. Second, since the Winograd algorithm changes the calculation unit of the neural network, the dataflow needs to be updated to accommodate changes in the calculation method.

There are also dataflows of CNNs for video applications [10], [11]. These dataflows can be expanded to make use of the similarity between frames, as shown in Fig. 5, and they provide a basis for our design. With the characteristics of the Winograd algorithm in mind, we have designed a dataflow that is more suitable for our accelerator. This dataflow helps to ensure the efficiency of our computing module.

The YOLOv3-tiny includes two operators: conv3 × 3 and conv1 × 1. When the Winograd algorithm is used to complete the operation of conv3 × 3, conv3 × 3 becomes a process with 4 × 4 matrix elementwise multiplication as the basic operation unit. As a result, there is no concept of a convolution sliding window of conv3 × 3. The kernel size of conv1 × 1 is 1 × 1, which can be alternatively viewed as matrix elementwise multiplication. This means that conv1 × 1 can be converted to a process with 4 × 4 matrix elementwise multiplication as the basic unit of operation. After this conversion, the dataflow of conv1 × 1 is like that of the conv3 × 3 operation under the Winograd algorithm. To summarize, our loops in our dataflow include frame, output channel, and input channel loops, along with the matrix position of the image loop.

In the dataflow for the conv3 × 3 convolutional layer (see Algorithm 1), the input channel index is ic, and the output channel index is oc. $W_{\mathrm{oc,ic}}$ is a 3 × 3 weight kernel unit corresponding to different input and output channels. Input $X^{\mathrm{ic}}_{h,w}$ is a 4 × 4 activation block reflecting the difference between two frames. h and w represent the position of the top-left pixel of the 4 × 4 activation value matrix on the image. B, G, and A are the conversion matrices in the Winograd algorithm. The related introduction of these matrices is detailed in Section II-B. $M[H/2][W/2]$ stores the number of

Algorithm 1 Pseudocode of Dataflow for conv3 × 3

Input:
The 3x3 weight kernel: $\boldsymbol{W_{oc,ic}}$;
The input differenced 4x4 activation matrix: $\boldsymbol{X^{ic}_{h,w}}$;
The Winograd transform matrix: ***B, G, A***;
The frame package batch size: **IG**;
The output feature map size: **H×W×OC**;
The index array: $\boldsymbol{Index[H/2][W/2][]}$
The nonzero differenced activation matrix channel: $\boldsymbol{M[H/2][W/2]}$
saved data:
The saved 4x4 activation matrix: $\boldsymbol{Y'^{oc}_{h,w}}$;
output:
The output 4x4 activation matrix: $\boldsymbol{Y^{oc}_{h,w}}$;

```
for oc = 0; oc < OC; oc=oc + N do
  for h = 0; h < H-4; h = h + 2 do
    for w = 0; w < W-4; w = w + 2 do
      for im_i = 0; im_i < IG; image_id++ do
        for s_oc = 0; s_oc < N; s_oc++ do
          if im_i == 0 then
            Y'^{oc+s_oc}_{h,w} = 0;
            Y^{oc+s_oc}_{h,w} = 0;
          end
          else then
            Y^{oc+s_oc}_{h,w} = Y'^{oc+s_oc}_{h,w};
          end
          for ic = 0; ic < M[h/2][w/2]; ic++ do
            ic_id = Index[h/2][w/2][ic];
            W_t_{oc,ic_id} = G W_{oc,ic_id} G^T;
            X_t^{ic_id}_{h,w} = B^T X^{ic_id}_{h,w} B;
            Y^{oc+s_oc}_{h,w} = Y^{oc+s_oc}_{h,w} + A^T [X_t^{ic_id}_{h,w} .
                                W_t_{oc,ic_id}] A;
          end
          Y'^{oc+s_oc}_{h,w} = Y^{oc+s_oc}_{h,w};
        end
      end
    end
  end
end
```

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

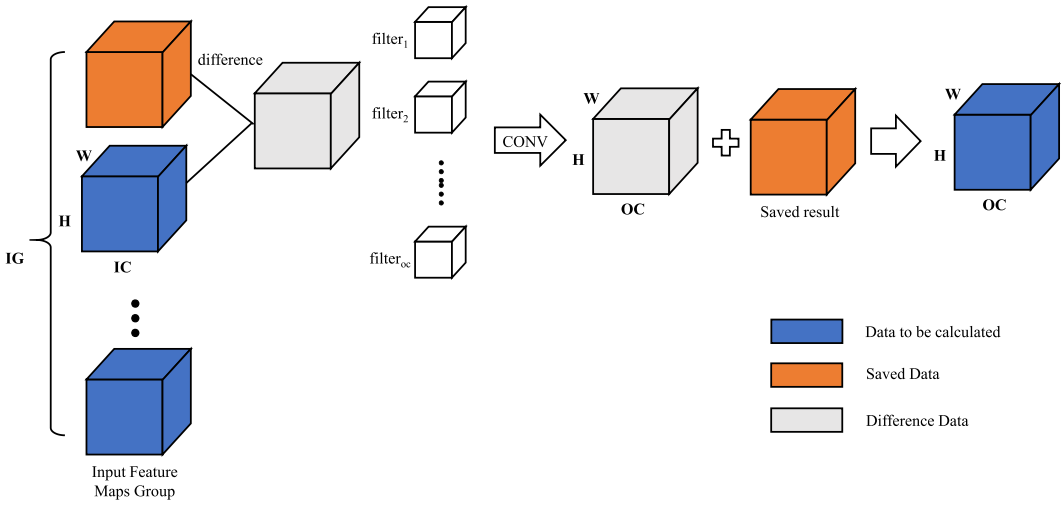


Fig. 5. Dataflow process with incremental operation.

channels of nonzero 4 × 4 activation blocks at each position. Incremental operation needs to be computed on each layer, so we process the data from all images in one image group at a time. The IG is the batch size of a frame package. Since the first frame of each package requires a complete convolution operation, the performance is different for different IGs. This will be discussed further in the following.

In order to facilitate the parallel hardware operation of the output channels, we put the loop over N output channels in the inner layer. For each channel, the saved reference convolution result $Y'^{oc+s_oc}_{h,w}$ is set to zero for the first frame of the frame package. For all other frames, the result is computed by adding the current convolution to the reference convolution. The innermost part of the dataflow loop is the matrix operation of the Winograd algorithm, including both weight and activation value matrix conversions along with the convolution result calculation. Each Winograd operation calculates the result of a 4 × 4 image block for one input channel, and the results from different input channels are added to the output result $Y^{oc+s_oc}_{h,w}$ until all N output channels have been processed. The saved reference convolution result $Y'^{oc+s_oc}_{h,w}$ is then updated to $Y^{oc+s_oc}_{h,w}$ calculated in the current iteration. The convolution of the next frame is performed, followed by the convolution of the image block at the next position until all N output channels are completed. Finally, the calculation of the next N output channels is performed until all results are calculated. Once the convolution calculation is complete, nonlinear calculations, such as ReLU and Maxpooling, are performed.

We changed the traditional conv1 × 1 dataflow (see Algorithm 2) to make it more like the conv3 × 3 dataflow used in the Winograd algorithm. Since the conv1 × 1 convolution does not use the Winograd algorithm, the input weight is expanded from the 1 × 1 kernel to the 4 × 4 matrix, and all elements in the matrix are the same. This is done in order to maintain the 4 × 4 matrix elementwise multiplication. Without the Winograd matrix conversion, the convolution result can be obtained by directly doing matrix elementwise multiplication. Therefore, the dataflow for conv1 × 1 is like one for conv3 × 3 using the Winograd algorithm that lacks some operations. This convolutional layer similarity is beneficial for improving the operating efficiency of the hardware.

Algorithm 2 Pseudocode of Dataflow for conv1 × 1

Input:
The 4x4 matrix of 1x1 weight kernel: $\boldsymbol{W_{oc,ic}}$;
The input differenced 4x4 activation matrix: $\boldsymbol{X^{ic}_{h,w}}$;
The frame package batch size: **IG**;
The input channel: **IC**;
The output feature map size: **H**×**W**×**OC**;
saved data:
The saved 4x4 activation matrix: $\boldsymbol{Y'^{oc}_{h,w}}$;
output:
The output 4x4 activation matrix: $\boldsymbol{Y^{oc}_{h,w}}$;

```
for oc = 0; oc < OC; oc=oc + N do
  for h = 0; h < H-4; h = h + 4 do
    for w = 0; w < W-4; w = w + 4
      for im_i = 0; im_i < IG; image_id++ do
        for s_oc = 0; s_oc < N; s_oc++ do
          if im_i == 0 then
            Y'^{oc+s_oc}_{h,w} = 0;
            Y^{oc+s_oc}_{h,w} = 0;
          end
          else then
            Y^{oc+s_oc}_{h,w} = Y'^{oc+s_oc}_{h,w};
          end
          for ic = 0; ic < IC; ic++ do
            Y^{oc+s_oc}_{h,w} = X_t^{ic}_{h,w} · W_t_{oc,ic};
          end
          Y'^{oc+s_oc}_{h,w} = Y^{oc+s_oc}_{h,w};
        end
      end
    end
  end
end
```

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

In order to choose dataflow parameters, including the number of frames in a frame package, N is set to the number of parallel output channels. Based on the algorithm, frame package size will have little effect on the accuracy of the convolution result. Since the first frame of each frame package performs a complete convolution operation, larger frame package sizes lead to higher utilization of interframe similarity and improved computational efficiency. However, frame package size will be limited by available BRAM resources since activation value data for each frame package must be partially stored there. Based on the BRAM resources on the ZCU104 board, we chose a frame package size of 8. The effect of similarity between frames on utilization and the acceleration efficiency of different frame package sizes will be discussed in detail in the following. The parallel efficiency of the accelerator is directly proportional to the number of parallel output channels, and the number of parallel output channels needs to match the off-chip bandwidth. Considering the bandwidth and on-chip resources, the number of parallel output channels is set to 16 in our design. The PE array of our design can perform 256 multiplications per clock cycle.

The dataflow and parallel mode designs are closely related to the hardware design. To improve the parallel efficiency of the hardware, we performed parallel operations on N output channels and 4×4 elementwise matrix multiplication. We attempted to ensure the use of differential input sparsity to achieve the best possible acceleration effect. A single processing unit can complete a 4×4 elementwise matrix multiplication operation, and our calculation array has N such units. Each unit is composed of 16 PEs, thus allowing the completion of the 4×4 elementwise matrix multiplication in one clock cycle. To meet the requirements of the Winograd algorithm, we also designed modules for Winograd matrix conversion. These modules are mainly composed of shifters and adders, which consumes very few hardware resources. Internal BRAM stores some of the weights and activation value data along with the convolution for the reference frame. The specific hardware structure will be introduced in detail in the following.

IV. Architecture of Accelerator

This section introduces the accelerator architecture in detail. We explain how we used the interframe similarity in hardware and how we deployed the Winograd algorithm.

Fig. 6 shows the overall structure of the CNN accelerator, which includes I/O, storage, control, and processing modules. The I/O module exchanges data with external DRAM using the AXI4 protocol, and the storage module includes input, output, and reference result buffers (RRBs). The processing module includes index and matrix transform modules, the process unit (PU) array, and correction, ReLU and maxpooling, and difference modules. The accelerator uses the PU array to perform elementwise multiplication for several output channels. The control module receives parameters regarding the current convolutional layer from the I/O module and then determines whether the PU should perform a conv1 $\times$ 1 or conv3 $\times$ 3 operation. It can also control processing loops based on the parameters of this layer.

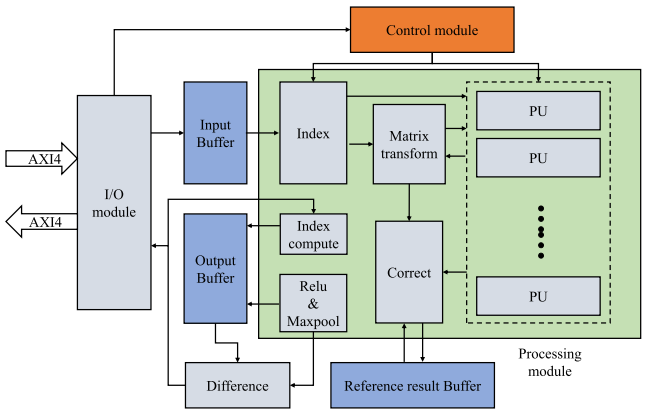


Fig. 6. Accelerator architecture.

A. Design for Skipping Similarity Between Frames

We designed many modules to use the similarity between frames in video applications. Modules related to incremental operation include the index, correction, index compute, and difference modules.

As mentioned earlier, we leverage the similarity between frames by subtracting the current frame from the reference frame. Upon the completion of the convolution operation, the reference frame is corrected to obtain the current frame. In order to skip input that does not need to be processed, the index module reads the nonzero activation block index from the input buffer and outputs the activation and weights of channels needing to be processed to subsequent modules. The correction model is used to get the convolution result of the current frame using the reference result. The module retrieves reference data from the RRB, produces the current frame by accumulating it with the differential convolution result, and updates the reference data in the RRB. In addition, we designed the index compute module, which calculates the nonzero activation block index of the next layer input. The difference module calculates the difference between the previous and current frames.

1) Index Module: In order to skip activation blocks that do not need to be calculated, we must process the data before performing the convolution operation. Since our differential input is densely stored, information that does not contribute to the result still consumes computing resources. To address this issue, the index module selects the data to be calculated using the input nonzero activation block index, and it outputs the selected data to the convolution processing module.

As shown in Fig. 7, 16 input channels of 4×4 activation value blocks are input every clock cycle. The nonzero activation block index stores the indexes of the N channels to be processed ($N < 16$), and the activation and corresponding weights of the N channels are selected in the next cycle. Using this approach, the convolution processing module will only receive data that need to be calculated. Channels that should be ignored are discarded in the index module, thus allowing our calculation to skip the similar data between frames.

When processing the first frame of each frame package, the input is the complete frame since the initial saved frame is zero. To ensure consistent operation between the first and

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

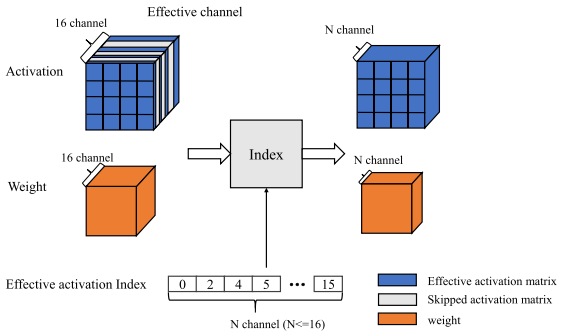


Fig. 7. Index processing.

subsequent frames, we set the first frame index to be dense, so no channels are discarded. This ensures the consistency of the calculation process and reduces the complexity of the hardware.

2) Correct Module: The convolution of the differential input is not the final result, but the difference between the current frame and the reference frame. The neural network contains both linear and nonlinear operations. Nonlinear operations cannot be performed as incremental operations, so we need a complete convolution result to run them. In the correct module, each time we get a set of convolution results, the reference frame convolution of the corresponding channel is combined with the result to get the current frame. The current frame is then stored back into the RRB. Since this process is pipelined, we can process the current set of data while calculating the next set of data.

The first frame contains the complete convolution result, so we directly overwrite the corresponding position in the RRB.

3) Index Compute Module: The index compute module accepts data from the difference module and calculates the nonzero index of that data. As shown in Fig. 8, the index compute module calculates the $L1$ norm for the 4×4 activation blocks for all 16 output channels. When the $L1$ norm of the differenced data is greater than our threshold, the activation value block needs to be processed and its channel number is added to the nonzero activation block index. Conversely, if the $L1$ norm is less than the threshold, we conclude that the activation value block does not contribute to the convolution result, and the channel number of the activation value block is omitted from the nonzero activation block index. This results in the activation block being skipped during the calculation of next layer.

We do not perform the threshold process for the first frame of each position frame package, so all activation value blocks are processed.

B. Design for Winograd Algorithm

The architecture of the Winograd convolution is shown in Fig. 9. The conv3 $\times$ 3 convolutional layer in the neural network uses the Winograd algorithm, and we adapted the convolution operation module accordingly. The convolution processing module for the Winograd algorithm includes a matrix transform module and PU array. The matrix transform module is used in the conv3 $\times$ 3 convolutional layer to convert input activation, input weights, and elementwise multiplication results. The PU array performs elementwise matrix multiplication operations. There are a total of 16 PU groups.

1) Matrix Transform Module: The matrix transform module includes the activation, weight, and elementwise multiplication result transform submodules. The activation transform computes GgG^T, the weight transform computes B^TdB, and the elementwise multiplication result transform computes $A^T RA$, where R is the phase of GgG^T and B^TdB. It is worth noting that although the matrix transform is matrix multiplication, the A, G, and B matrix elements only contain 1, 0.5, and 2 as values, respectively. This means that we do not need a hardware multiplication unit to perform transformations, and we can use shifters and adders. Since there is no need to perform a matrix transform when computing conv1 $\times$ 1 convolutional layers, the matrix transform module is not active during this phase. The input data width is 8 bit, and the transformed data width is 10 bit. Before performing the elementwise matrix multiplication, we truncate the results. The output data width is 8 bit.

2) PU Array: In our accelerator, the PU array is responsible for performing elementwise matrix multiplication operations, and it will calculate the results of 16 output channels every clock cycle. Each PU receives the activation and weights from either the matrix transform or index module, and they all receive the same activation and weights of different output channels; 16 PEs are included in each PU to ensure that 16 multiply-accumulate operations can be completed in one clock cycle. A PE performs multiplication and accumulation operations, and it output the results after all the required input channels have been calculated. A PE has an 8 bit $\times$ 8 bit multiplier. The structure of the PU array is shown in Fig. 10.

C. Storage

As shown in Fig. 11, storage is divided into input, output, and RRBs. The input and output buffers can be further subdivided into the input index, weight and input activation buffers (IMBs), and the output index and output activation buffers (OMBs). Each buffer has a ping–pong structure to improve processing efficiency, and this is described later in this article. All activations and weights have been quantized to 8-bit fixed-point numbers.

The IMB includes ping and pong buffers. The width of each buffer is 128 bits, the depth is 196, and the size of each ping–pong buffer is 3136 bytes. Each IMB can store 16 channels of 14×14 activation blocks. For the conv3 $\times$ 3 convolutional layer, the IMB is fully utilized, and the 14×14 activation block contains a 12×12 result after convolution. In order to maintain the consistency of the output activation with the conv1 $\times$ 1 layer, the IMB may only use 12 bit $\times$ 12 bit $\times$ 128 bit data. For conv3 $\times$ 3, if the complete input activation is greater than 14×14, it is divided into several 14×14 activation blocks, where each activation block has a 2×14 block overlap. Likewise, for conv1 $\times$ 1, if the complete input activation size is greater than 12×12, the input

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

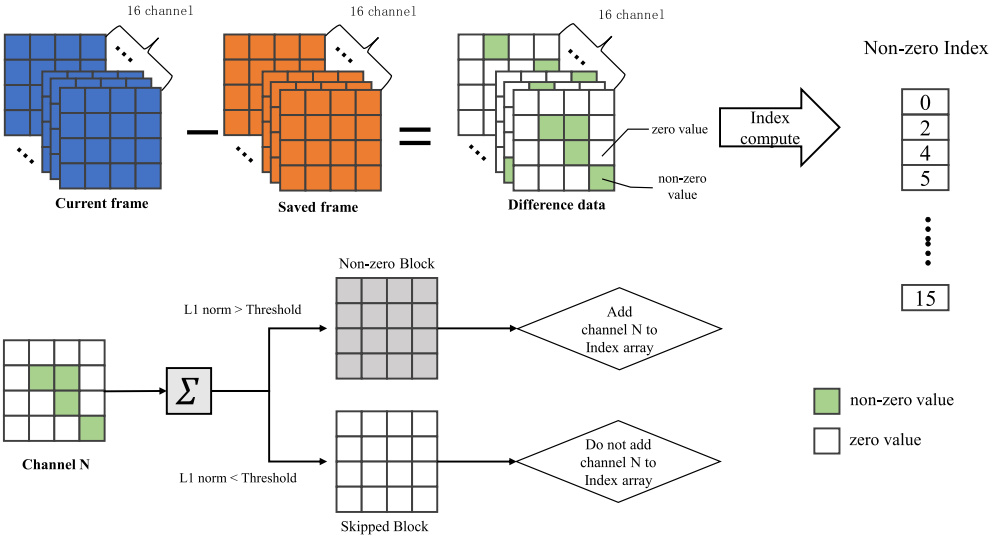


Fig. 8. Index compute process.

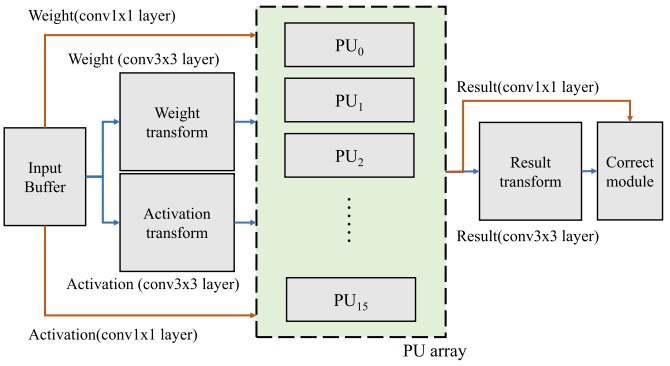


Fig. 9. Architecture of the Winograd convolution.

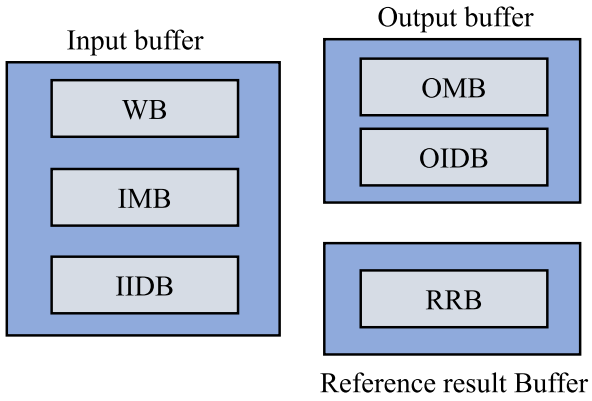


Fig. 11. Structure of storage.

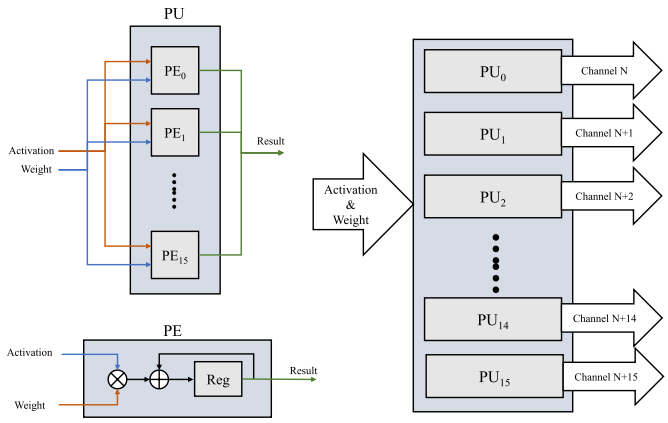


Fig. 10. Structure of PU array.

activation is divided into several 12 × 12 activation blocks. The input weight buffer (WB) also contains ping and pong buffers. The bit width of each buffer is 128 bits, the depth is 144, and the size of each ping–pong buffer is 2304 bytes. Each WB can store up to 16 input channels corresponding to the weights in the 3 × 3 convolution kernel of 16 output channels. For conv1 × 1, we only use 256 bytes.

The input index buffer (IIDB) stores the index of the nonzero 4 × 4 activation block, its size is 576 bytes, and the ping–pong structure is also used. The index of each position stores the serial number of the nonzero channels in the block.

The OMB includes ping and pong buffers. It is 24 bits wide, so the width of each buffer is 24 bit × 16 bit = 384 bits, the depth is 144, and the size of each buffer is 6912 bytes. Each block of the OMB can store 12 × 12 activation blocks of 16 output channels. The maxpooling operation only uses 384 bits × 6 bit × 6 bit, and it stores 6 × 6 activation blocks of 16 output channels. When the maxpooling operation is not required, the output buffer is fully utilized. The I/O module shifts and truncates the 24-bit activation to transform it into an 8-bit activation and outputs it through the AXI bus.

The output index buffer (OIDB) is the same size as the IIDB, and it stores the nonzero 4 × 4 activation block index of the difference between the output of the current layer and the saved result.

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

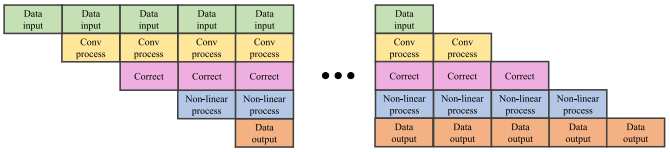


Fig. 12. Processing pipeline.

TABLE III
RESOURCE UTILIZATION

	DSP	FF	LUT	BRAM_18K	Power (W)
Utilized	146	45083	123359	326	3.301
Available	1728	460800	230400	624	
Percentage	8%	9%	53%	52%	

The ping–pong structure enables the input and output stages to be executed in parallel with the data processing stage. One buffer accepts the data from the I/O module, while the other is used for processing, which improves the efficiency of the hardware since a pipeline is produced. The pipeline of our design is shown in Fig. 12.

The RRB is 6912 bytes, and it stores the convolution of the previous frame, which is added to the differential convolution to obtain the convolution of the current frame. The frames are compared to determine the output index. After the current frame is completely processed, the data in the RRB are updated to the convolution result for the current frame.

V. EXPERIMENT

In this article, we used a Xilinx ZCU104 to evaluate our design. Vivado HLS 2019.2 was used to implement the accelerator and to generate the bitstream. The accelerator runs at a frequency of 200 MHz. We completed the performance evaluation of the YOLOv3-tiny network accelerator using an FPGA, and the power consumption was evaluated using the Xilinx Power Estimator.

A. Resource Utilization

The resource consumption of our accelerator on the FPGA development board is shown in Table III. The most important value is the utilization of the BRAM_18K and DSP resources. BRAM provides caching and storage in the accelerator. The memory size is determined by the frame package and stored input and output data channel sizes. Increasing frame package size provides higher speedup, and using more stored input and output data channels increases the multiplexing of input data. This reduces data exchanges between the accelerator and DDR. DSP is primarily used for multiply–add operations in the accelerator. The PU array performs 256 multiplication and addition operations per clock cycle. Since YOLOv3-tiny is 8-bit quantized, each of the 128 DSP resources implements two multiplication operations. Later, we will discuss the throughput and energy consumption provided by each DSP.

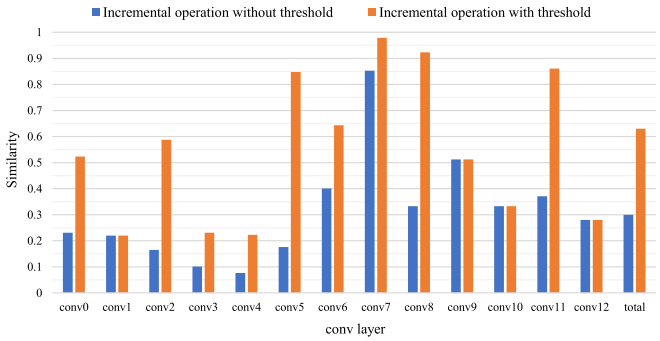


Fig. 13. Similarity between frames of different thresholds.

B. Arithmetic Complexity

1) Similarity Between Frames and Arithmetic Complexity: When determining if a 4×4 activation value block needs to be processed, we calculate the $L1$ norm and omit it if it is less than our threshold. We set different thresholds in different layers, corresponding to Table II.

The similarity between frames reflects the proportion of operations that can be skipped during incremental operations. The higher the similarity between frames, the more convolution operations that can be skipped, and the better the acceleration effect. The accuracy of the network is determined by the threshold. In order to discuss the changes in the similarity between frames using thresholds or not, we calculated the similarity between the input frames of each convolutional layer under thresholds of Table II and no threshold on the ImageNet ILSVRC2015 video dataset. This statistic reflects the average interframe similarity of all frames in a dataset relative to the previous frame. The results are shown in Fig. 13.

When we use incremental operation with no threshold, the average interframe similarity of network is 0.2998. When we use incremental operation with the thresholds of Table II, the average interframe similarity of network improves to 0.6297. The interframe similarity reflects the proportion of the convolution calculation that can be skipped for each frame in an ideal situation. Ideally, the arithmetic complexity of incremental operations with no threshold is 70.02%, and the speedup is $1.428\times$. With thresholds of Table II, the arithmetic complexity is 37.03%, and the speedup is $2.701\times$. The incremental operation with threshold reduces the arithmetic complexity and improves the speedup without accuracy loss. All the incremental operations we mentioned later will use the thresholds of Table II.

Recall that due to the limitation of hardware resources, the size of the frame package cannot be expanded indefinitely. The first frame of the frame package will perform a complete convolution operation instead of an incremental operation. Therefore, the size of the frame packet will affect the actual algorithm complexity. Besides, in actual hardware acceleration, a part of the data input and output time cannot be covered by the calculation time, so the actual speedup will be lower than the ideal speedup.

2) Frame Package Size and Arithmetic Complexity: The first frame of each frame package must do a complete convolution

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
NETWORK PROCESSING TIME

	input size	GFLOPs	original network (ms)	network with Winograd algorithm (ms)	network with Winograd algorithm & Incremental operation (ms)
conv0	384×384×3	0.139	54.74	25.24	14.45
conv1	192×192×16	0.345	27.37	12.62	10.35
conv2	96×96×32	0.343	29.39	14.64	8.57
conv3	48×48×64	0.341	28.14	13.4	11.01
conv4	24×24×128	0.340	28.12	13.38	11.07
conv5	12×12×256	0.340	27.96	13.22	4.46
conv6	12×12×512	1.360	109.08	50.1	23.55
conv7	12×12×1024	0.076	8.61	8.61	4.21
conv8	12×12×256	0.340	27.41	12.66	3.87
conv9	12×12×512	0.015	1.95	1.95	1.37
conv10	12×12×256	0.010	1.32	1.32	1.11
conv11	24×24×384	1.020	80.98	36.74	10.38
conv12	24×24×256	0.031	4.03	4.03	3.39
total	/	4.69	429.09	207.91	107.81

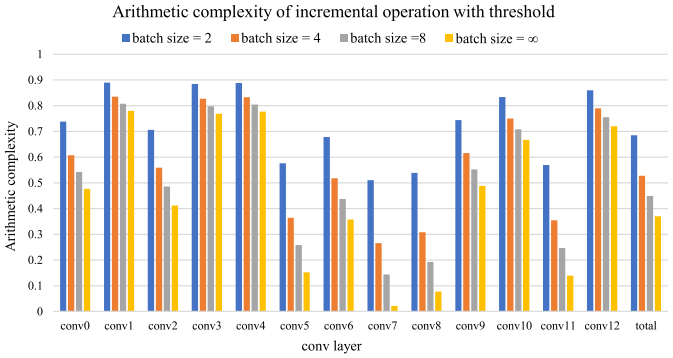


Fig. 14. Arithmetic complexity of different batch sizes.

operation, which reduces the utilization of similarity between frames. We define the size of each frame package as the frame batch size. The larger the frame package is, the fewer frames need to do the complete convolution operation, and the higher the utilization of the similarity between frames.

The utilization of interframe similarity is reflected in the arithmetic complexity of convolution operations. The arithmetic complexity is the ratio of the number of convolution operations to the number of original convolution operations. We evaluated the arithmetic complexity of different frame batch sizes on the ImageNet ILSVRC2015 dataset of the thresholds of Table II. The frame batch size was set to 2, 4, and 8. The result is shown in Fig. 14.

As the frame batch size increases, the arithmetic complexity of the network approaches the ideal arithmetic complexity. Under the thresholds of Table II, the ideal network arithmetic complexity is 37.03% of the original convolution. The network arithmetic complexity is 68.51%, 52.76%, and 44.89% for batch sizes 2, 4, and 8, respectively. This indicates that we should set the frame batch size as large as possible.

C. Performance

The accelerator frequency we designed is 200 MHz, and the frame batch size is 8. On the ImageNet ILSVRC2015 dataset, we evaluated the accelerator's processing time of the YOLOv3-tiny network on the original setup, using Winograd algorithm, and using Winograd algorithm and incremental operation. We observed the processing time of each layer, as well as the total processing time. The time required to perform ReLU and maxpooling processing after a convolutional layer was included in the processing time for that layer. The processing time for each layer and the total includes the network processing time to complete all eight frames of a frame package. The average network process time of all frame packages in dataset is shown in Table IV.

The processing time of the original network is about 429.1 ms. With the Winograd algorithm, the processing time of the network is roughly 207.9 ms. With the Winograd algorithm and incremental operation, the processing time of the network is about 107.8 ms. The Winograd algorithm and the utilization

TABLE V
PERFORMANCE OF ACCELERATOR IN THREE CASES

	quantized network	Winograd algorithm	Winograd algorithm & Incremental operation
Dataset	/	ImageNet ILSVRC2015	ImageNet ILSVRC2015
mAP	0.731	0.731	0.731
FPS	18.64	34.48	74.20
GOPs	87.44	180.46	348.02
Speedup	1x	2.06x	4.10x
DSP	146	146	146
kLUT	123.36	123.36	123.36
Power/w	3.301	3.301	3.301
GOPs/DSP	0.599	1.236	2.384
GOPs/kLUT	0.709	1.463	2.821
GOPs/w	26.49	54.67	105.43

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCE COMPARISON WITH PREVIOUS IMPLEMENTATION

	[28]	[18]	[19]	[20]	This work
year	2020	2020	2021	2021	2021
CNN type	R(2+1)D	YOLOv3-tiny	YOLOv2-tiny	YOLOv3-tiny	YOLOv3-tiny
Device	Xilinx ZCU102	Virtex-7 VC707	Xilinx ZC706	Ultra96 v2	Xilinx ZCU104
Dataset	UCF101	\	KITTI	Custom VOC 2007	ImageNet ILSVRC2015
mAP	\	\	0.770	0.75	0.731
mAP Drop	\	\	\	2.0%	1.1%
Frequency(MHz)	150	200	200	250	200
Precision	16-bit fixed	18-bit fixed	8-bit fixed	8-bit fixed	8-bit fixed
DSP Utilization	695(28%)/1215(48%)	2304(82%)	610(68%)	242(67%)	146(8%)
kLUT Utilization	74(27%)/148(54%)	49(16%)	84(38%)	27(39%)	123(53%)
Throughput/GOPS	67.7/111.7	460.8	464.5	31.5	348.0
Power(w)	5.4/6.7	4.81	10.25	4.26	3.301
GOPS/DSP	0.097/0.092	0.200	0.761	0.130	2.384
GOPS/kLUT	0.915/0.755	9.48	5.52	1.667	2.821
Normalized DSP efficiency	0.065/0.061	0.100	0.190	0.026	0.596
GOPS/w	12.5/16.7	95.8	45.3	7.40	105.4

of the similarity between frames greatly reduce the network processing time of each frame package.

The Winograd algorithm reduces the number of multiplications of the conv3 × 3 layers, making the processing time of conv3 × 3 layers shorter. Compared to the original network, the network processing time with the Winograd algorithm has a speedup of 2.06×. The incremental operation allows each layer to skip part of the convolution operation, providing 1.99× speedup. The network using the Winograd algorithm and incremental operation has 4.10× speedup compared to the original network.

Through the processing time and the GFLOPs of the YOLOv3-tiny network, we calculated the performance of the accelerator in three cases, as shown in Table V. In general, our design provides 4.1× speedup with no loss of accuracy.

The design of our algorithm will not be affected by the hardware structure. Regardless of the size of the hardware structure, the same proportion of convolution can still be skipped. Therefore, the architecture with different sizes can still obtain the same relative speedup.

D. Result Comparison

As a result, we recorded the performance of our design using general video applications (ImageNet ILSVRC2015 dataset). We then compared the performance with a 3-D CNN accelerator [28] and other YOLO accelerators [18]–[20]. The comparison data are shown in Table VI.

One 25 bit × 18 bit DSP can be decomposed to handle two 8 bit × 8 bit multiplications, while it can only handle one 16 bit × 16 bit or 18 bit × 18 bit multiplication. Thus, 8-bit accelerator has twice overall DSP efficiency compared with 16-bit accelerator. Also, the working frequency is in part determined by the occupation of the FPGA device. Thus, the normalized DSP efficiency of different designs can be computed as follows:

$$\text{Normalized DSP efficiency} = \frac{\text{DSP efficiency}}{\alpha \times (f \div 100\ \text{MHz})}. \quad (9)$$

Here, $\alpha = 1$ for 16-bit or 18-bit accelerator and $\alpha = 2$ for 8-bit accelerator.

The DSP efficiency of our design is 2.384 GOPS/DSP. The normalized DSP efficiency is 0.596. Compared to the 3-D CNN accelerator, our design can provide a 9.17×–9.77× normalized DSP efficiency. Compared to other YOLO accelerators, our design can provide a 3.13×–18.34× normalized DSP efficiency.

Our design has an energy efficiency of 105.4 GOPS/w. Compared to other designs, our design provides 1.10×–14.2× energy efficiency.

In general, due to video optimizations, the CNN accelerator we designed has significant advantages over other neural network accelerators in video applications.

VI. CONCLUSION

In this article, we proposed a CNN FPGA accelerator that takes advantage of the similarity between frames in video applications. The accelerator skips the calculation of similar data between frames by using incremental operations. In addition, we used the Winograd algorithm to improve the utilization of DSP resources on the FPGA while increasing accelerator throughput. We designed a dataflow based on the characteristics of the incremental operation and Winograd algorithms, which increased the efficiency of data reuse and array processing. Experimental results show that our accelerator achieved 74.2 frames/s on ImageNet ILSVRC2015. Compared to the original network without Winograd algorithm and incremental operation, our design provides a 4.10× speedup. When compared with other YOLO network FPGA accelerators applied to video applications, our design provided

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.

a 3.13×–18.34× normalized DSP efficiency and 1.10×–14.2× energy efficiency.

ACKNOWLEDGMENT

The authors would like to thank LetPub (www.letpub.com) for its linguistic assistance during the preparation of this manuscript.

REFERENCES

[1] C.-H. Lin *et al.*, "7.1 A 3.4-to-13.3TOPS/W 3.6TOPS dual-core deep-learning accelerator for versatile AI applications in 7 nm 5G smartphone SoC," in *IEEE ISSCC Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 134–136, doi: 10.1109/ISSCC19947.2020.9063111.

[2] Y. Jiao *et al.*, "7.2 A 12 nm programmable convolution-efficient neural-processing-unit chip achieving 825TOPS," in *IEEE ISSCC Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 136–140, doi: 10.1109/ISSCC19947.2020.9062984.

[3] W. Shan *et al.*, "14.1 A 510 nW 0.41 V low-memory low-computation keyword-spotting chip using serial FFT-based MFCC and binarized depthwise separable convolutional neural network in 28 nm CMOS," in *IEEE ISSCC Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 230–232, doi: 10.1109/ISSCC19947.2020.9063000.

[4] P. C. Knag *et al.*, "A 617 TOPS/W all digital binary neural network accelerator in 10 nm FinFET CMOS," in *Proc. IEEE Symp. VLSI Circuits*, Honolulu, HI, USA, Jun. 2020, pp. 1–2, doi: 10.1109/VLSICircuits18222.2020.9162949.

[5] J.-W. Chang, K.-W. Kang, and S.-J. Kang, "SDCNN: An efficient sparse deconvolutional neural network accelerator on FPGA," in *Proc. Design, Automat. Test Eur. Conf. Exhib. (DATE)*, Florence, Italy, Mar. 2019, pp. 968–971, doi: 10.23919/DATE.2019.8715055.

[6] J.-W. Chang, K.-W. Kang, and S.-J. Kang, "An energy-efficient FPGA-based deconvolutional neural networks accelerator for single image super-resolution," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 1, pp. 281–295, Jan. 2020, doi: 10.1109/TCSVT.2018.2888898.

[7] L. Lu, J. Xie, R. Huang, J. Zhang, W. Lin, and Y. Liang, "An efficient hardware accelerator for sparse convolutional neural networks on FPGAs," in *Proc. IEEE 27th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, San Diego, CA, USA, Apr. 2019, pp. 17–25, doi: 10.1109/FCCM.2019.00013.

[8] Z. Wang, K. Xu, S. Wu, L. Liu, L. Liu, and D. Wang, "Sparse-YOLO: Hardware/software co-design of an FPGA accelerator for YOLOv2," *IEEE Access*, vol. 8, pp. 116569–116585, 2020, doi: 10.1109/ACCESS.2020.3004198.

[9] H. Kim and K. Choi, "Low power FPGA-SoC design techniques for CNN-based object detection accelerator," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, New York, NY, USA, Oct. 2019, pp. 1130–1134, doi: 10.1109/UEMCON47517.2019.8992929.

[10] Z. Yuan *et al.*, "14.2 A 65 nm 24.7 μJ/frame 12.3 mW activation-similarity-aware convolutional neural network video processor using hybrid precision, inter-frame data reuse and mixed-bit-width difference-frame data codec," in *IEEE ISSCC Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 232–234, doi: 10.1109/ISSCC19947.2020.9063155.

[11] M. Riera, J.-M. Arnau, and A. Gonzalez, "Computation reuse in DNNs by exploiting input similarity," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Los Angeles, CA, USA, Jun. 2018, pp. 57–68, doi: 10.1109/ISCA.2018.00016.

[12] M. Buckler, P. Bedoukian, S. Jayasuriya, and A. Sampson, "EVA: Exploiting temporal redundancy in live computer vision," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Los Angeles, CA, USA, Jun. 2018, pp. 533–546, doi: 10.1109/ISCA.2018.00051.

[13] Y. Wang, Y. Wang, H. Li, Y. Han, and X. Li, "An efficient deep learning accelerator for compressed video analysis," in *Proc. 57th ACM/IEEE Design Automat. Conf. (DAC)*, San Francisco, CA, USA, Jul. 2020, pp. 1–6, doi: 10.1109/DAC18072.2020.9218743.

[14] C. Zhu, K. Huang, S. Yang, Z. Zhu, H. Zhang, and H. Shen, "An efficient hardware accelerator for structured sparse convolutional neural networks on FPGAs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 9, pp. 1953–1965, Sep. 2020.

[15] S. Zhang, J. Cao, Q. Zhang, Q. Zhang, Y. Zhang, and Y. Wang, "An FPGA-based reconfigurable CNN accelerator for Yolo," in *Proc. IEEE 3rd Int. Conf. Electron. Technol. (ICET)*, Chengdu, China, May 2020, pp. 74–78, doi: 10.1109/ICET49382.2020.9119500.

[16] Q. Yin *et al.*, "FPGA-based high-performance CNN accelerator architecture with high DSP utilization and efficient scheduling mode," in *Proc. Int. Conf. High Perform. Big Data Intell. Syst. (HPBD&IS)*, Shenzhen, China, May 2020, pp. 1–7, doi: 10.1109/HPBDIS49115.2020.9130576.

[17] D. T. Nguyen, T. N. Nguyen, H. Kim, and H. J. Lee, "A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 8, pp. 1861–1873, Aug. 2019.

[18] A. Ahmad, M. A. Pasha, and G. J. Raza, "Accelerating tiny YOLOv3 using FPGA-based hardware/software co-design," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Seville, Spain, Oct. 2020, pp. 1–5, doi: 10.1109/ISCAS45731.2020.9180843.

[19] J. Zhang *et al.*, "A low-latency FPGA implementation for real-time object detection," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Daegu, South Korea, May 2021, pp. 1–5, doi: 10.1109/ISCAS51556.2021.9401577.

[20] T. Adiono, A. Putra, N. Sutisna, I. Syafalni, and R. Mulyawan, "Low latency YOLOv3-tiny accelerator for low-cost FPGA using general matrix multiplication principle," *IEEE Access*, vol. 9, pp. 141890–141913, 2021, doi: 10.1109/ACCESS.2021.3120629.

[21] P. O'Connor and M. Welling, "Sigma delta quantized networks," 2016, *arXiv:1611.02024*.

[22] D. Neil, J. H. Lee, T. Delbruck, and S.-C. Liu, "Delta networks for optimized recurrent network computation," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, Sydney, NSW, Australia, 2017, pp. 2584–2593.

[23] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 4013–4021, doi: 10.1109/CVPR.2016.435.

[24] X. Wang, C. Wang, J. Cao, L. Gong, and X. Zhou, "WinoNN: Optimizing FPGA-based convolutional neural network accelerators using sparse Winograd algorithm," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 4290–4302, Nov. 2020, doi: 10.1109/TCAD.2020.3012323.

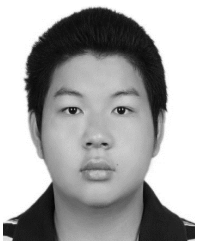
[25] J. Yepez and S.-B. Ko, "Stride 2 1-D, 2-D, and 3-D Winograd for convolutional neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 4, pp. 853–863, Apr. 2020, doi: 10.1109/TVLSI.2019.2961602.

[26] B. Widrow, I. Kollar, and M.-C. Liu, "Statistical theory of quantization," *IEEE Trans. Instrum. Meas.*, vol. 45, no. 2, pp. 353–361, Apr. 1996, doi: 10.1109/19.492748.

[27] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Seoul, South Korea, Jun. 2016, pp. 367–379, doi: 10.1109/ISCA.2016.40.

[28] M. Sun, P. Zhao, M. Gungor, M. Pedram, M. Leeser, and X. Lin, "3D CNN acceleration on FPGA using hardware-aware pruning," in *Proc. 57th ACM/IEEE Design Automat. Conf. (DAC)*, San Francisco, CA, USA, Jul. 2020, pp. 1–6, doi: 10.1109/DAC18072.2020.9218571.

[29] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015, doi: 10.1007/s11263-015-0816-y.



Shengzhao Li (Graduate Student Member, IEEE) received the B.S degree from Shanghai Jiao Tong University, Shanghai, China, in 2019, where he is currently working toward the master's degree at the College of Electronic Engineering.

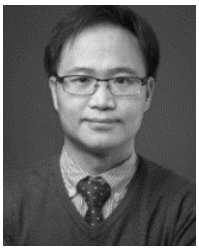
His research interest includes hardware acceleration of neural networks.



Qin Wang (Member, IEEE) received the B.S. degree from the University of Electronics Science and Technology of China, Chengdu, China, in 1997, and the Ph.D. degree from Shanghai Jiao Tong University, Shanghai, China, in 2004.

She is currently a Professor with the Department of Micro/Nano Electronics, Shanghai Jiao Tong University. Her research interests include high-performance processor and in-memory computing.

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.



Jianfei Jiang (Member, IEEE) received the B.S. degree from Zhejiang University, Hangzhou, China, in 2000, and the M.S. and Ph.D. degrees from Shanghai Jiao Tong University, Shanghai, China, in 2007 and 2017, respectively.

He is currently an Assistant Professor with the Department of Microelectronics and Nanoscience, Shanghai Jiao Tong University. His current research interests include high-speed on-chip interconnect, low-power circuit design, and high-speed circuit design.



Weiguang Sheng (Member, IEEE) received the bachelor's, master's, and Ph.D. degrees from the Harbin Institute of Technology, Harbin, China, in 1999, 2004, and 2009, respectively.

He is currently a Research Assistant Professor with the Department of Micro/Nano Electronics, Shanghai Jiao Tong University, Shanghai, China. His research interests include reconfigurable architectures and compiling techniques, soft error analysis, and optimization.



Naifeng Jing (Senior Member, IEEE) received the Ph.D. degree from Shanghai Jiao Tong University, Shanghai, China, in 2012.

He is currently an Associate Professor with the Department of Micro/Nano Electronics, Shanghai Jiao Tong University. His research interests include high-performance and high-reliability computing architecture and systems, in-memory computing architecture, and computer-aided VLSI design.



Zhigang Mao (Member, IEEE) received the B.S. degree from Tsinghua University, Beijing, China, in 1986, and the Ph.D. degree from the University of Rennes 1, Rennes, France, in 1992.

From 1992 to 2006, he was with the Microelectronics Center, Harbin Institute of Technology, Harbin, China. In 2006, he joined the Department of Micro/Nano Electronics, Shanghai Jiao Tong University, Shanghai, China, where he is currently a Professor. His current research interests include digital signal processor (DSP) architecture design, video processor design, and reconfigurable processor architecture.

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on September 03,2026 at 13:11:42 UTC from IEEE Xplore. Restrictions apply.